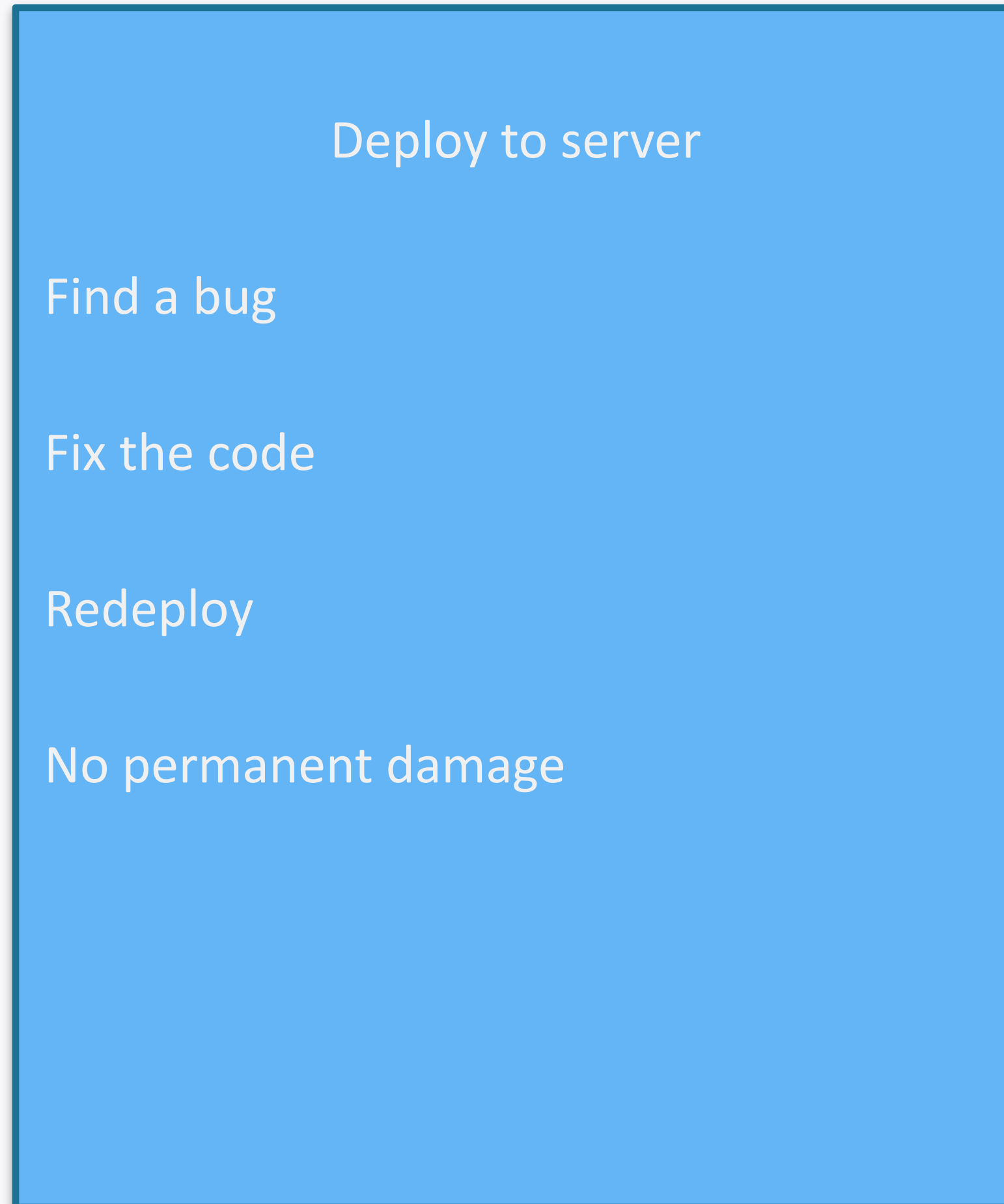


Module 5

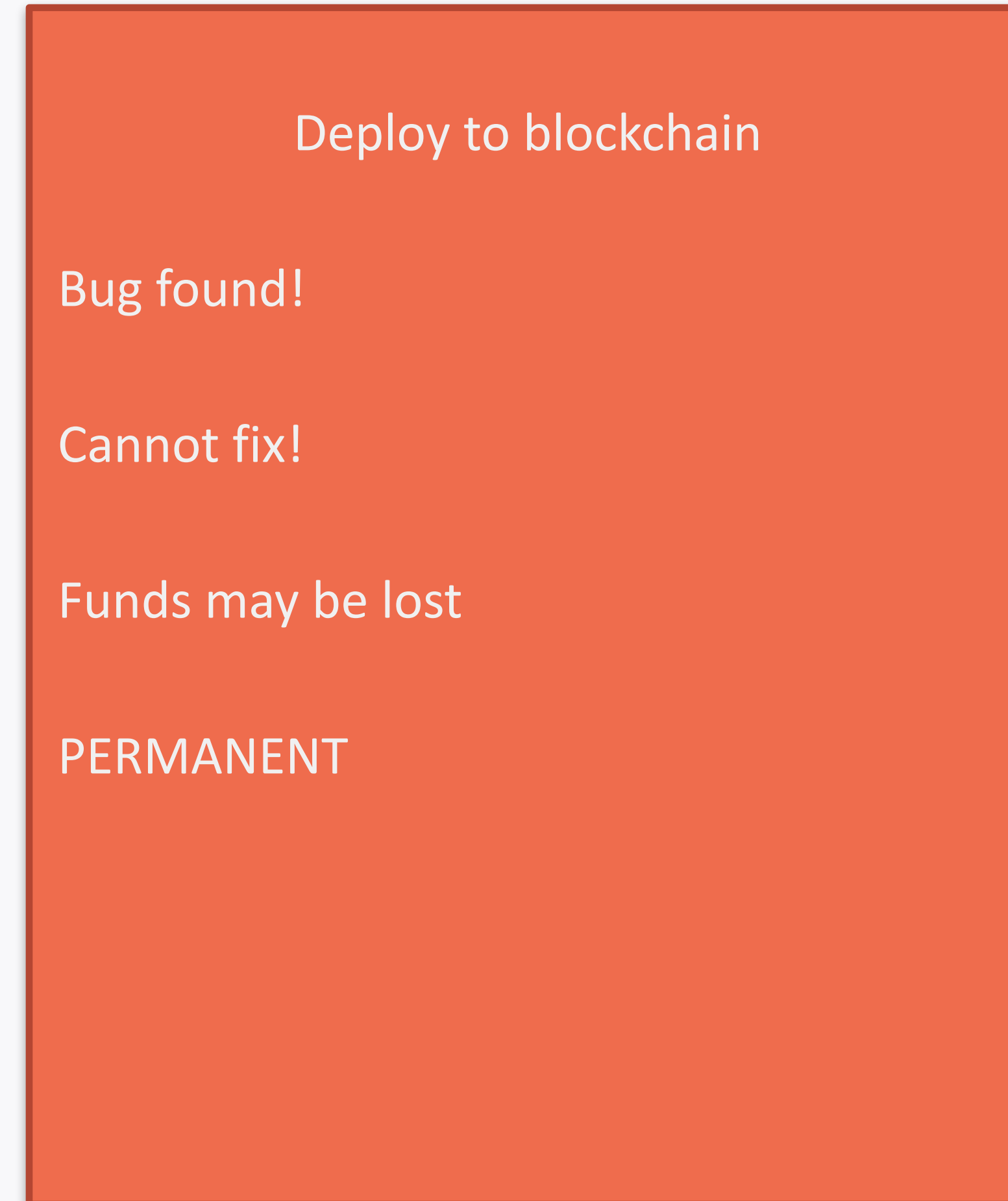
Testing Smart Contracts (Foundry)

Why Testing is Critical

Web2 Development



Ethereum Smart Contracts



Testing Philosophy in Web3

"If It's Not Tested, It's Broken"

In Traditional Software:

Deploy → Fix bug → Redeploy

Users experience downtime

Funds are safe

In Smart Contracts:

Deploy → Bug exists → PERMANENT

Users lose money → Forever

No way to patch

Testing is not optional.

It's insurance against catastrophic failure

Installing & Initializing Foundry

```
# Install foundry
curl -L https://foundry.paradigm.xyz
    | bash
foundryup

# Create new project
forge init my-project
cd my-project

# Project structure
src/    → Contracts
test/   → Tests (Solidity)
script/ → Deployments
```

One-line install

Initializes git repo

Structure ready for professional work

Example: Counter Contract

```
pragma solidity ^0.8.20;

contract Counter {
    uint256 public count;

    function increment()
        public {
            count += 1;
        }

    function decrement()
        public {
            require(count > 0,
                "Underflow");
            count -= 1;
        }
}
```

Simple state contract

Two public functions

decrement() has safety check

Writing Your First Test

```
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/Counter.sol";

contract CounterTest is Test {
    Counter counter;

    function setUp() public {
        counter = new Counter();
    }
}
```

Inherit from Test for assertions & vm

setUp() runs before each test

Fresh instance created

Basic Assertion Test

```
function testInitialValueIsZero()  
  public {  
    assertEquals(  
      counter.count(), 0  
    );  
  }  
  
function testIncrement() public {  
  counter.increment();  
  
  assertEquals(  
    counter.count(), 1  
  );  
}
```

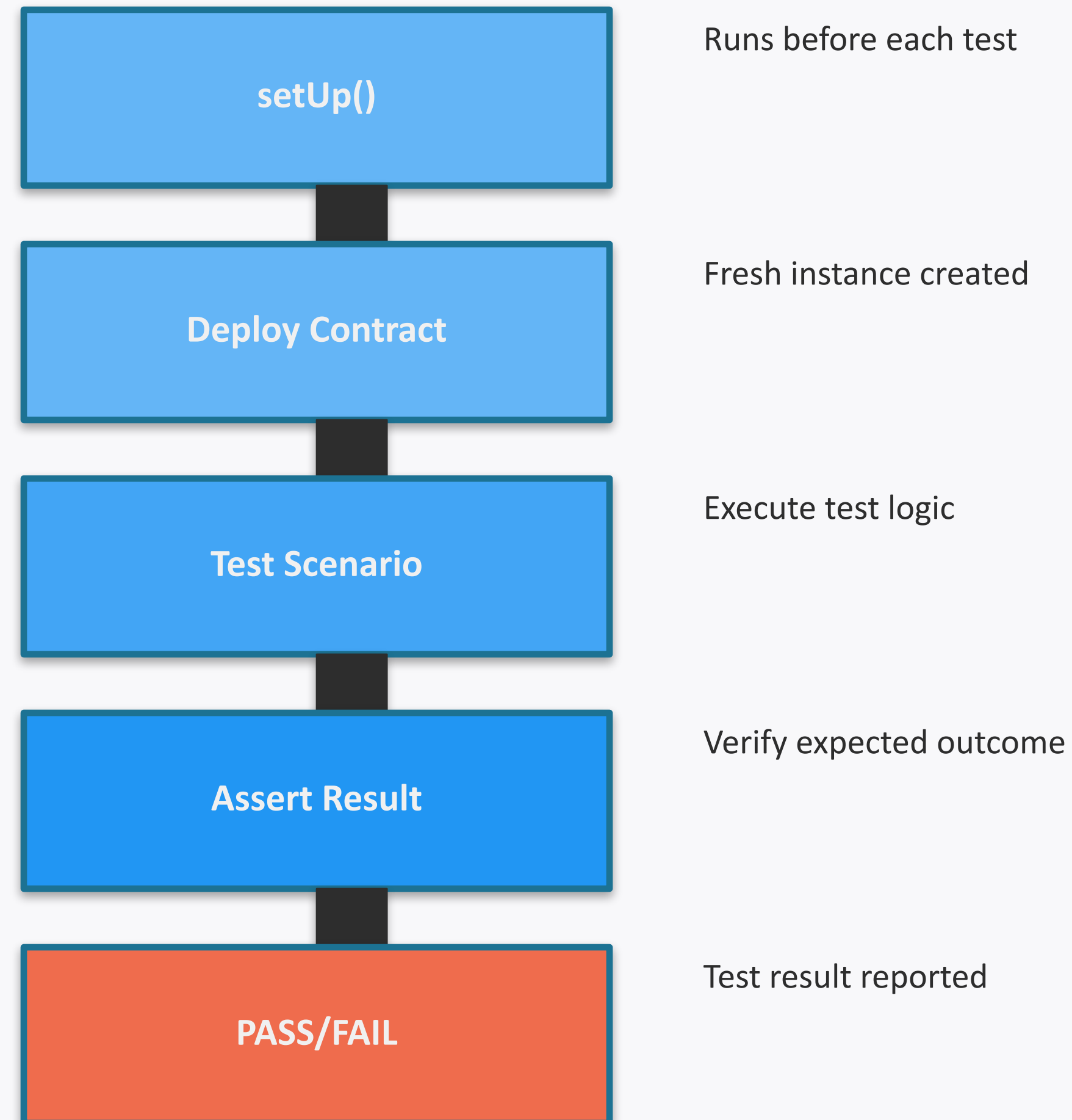
assertEquals() checks equality

Test name convention:

testXxx()

Runs in any order

Test Execution Flow



Testing Failure Cases

```
function testDecrementRevertsZero()  
  public {  
  
    vm.expectRevert(  
      "Underflow"  
    );  
  
    counter.decrement();  
  }
```

vm = cheatcode object

expectRevert() tells test:
'Next call should revert'

If it doesn't → FAIL

If it does → PASS

Testing Access Control

```
function testOnlyOwner()  
  public {  
  
    vm.prank(address(1));  
    // msg.sender = address(1)  
  
    vm.expectRevert(  
      "Not owner"  
    );  
  
    owned.restricted();  
  }
```

vm.prank() simulates different caller

Critical for testing multi-user scenarios

Highly useful pattern

Running Tests with Forge

Execute Tests:

```
$ forge test
```

Output Shows:

```
[PASS] testInitialValuelsZero()
```

```
[PASS] testIncrement()
```

```
[PASS] testDecrementRevertsZero()
```

View Gas Usage:

```
$ forge test --gas-report
```

Run Specific Test:

```
$ forge test --match-test testIncrement
```

Introduction to Fuzz Testing

```
function testFuzzTransfer(  
  uint256 amount  
) public {  
  vm.assume(amount <= 1000);  
  
  token.transfer(  
    address(1), amount  
  );  
  
  assertEq(  
    token.balanceOf(address(1)),  
    amount  
  );  
}
```

Parameter = input

Foundry generates many random values

Tests edge cases automatically

Why Fuzz Testing Matters

Finds Bugs Automatically

- Tests thousands of inputs
- You only write one test
- Covers boundary conditions

What Fuzzing Catches:

- Integer overflow/underflow
- Off-by-one errors
- Unexpected reverts
- Logic breaks with edge values

Professional Standard:

- Always include fuzz tests

Professional Testing Patterns

Test Structure:

- One assertion per concept
- Fresh contract in setUp()
- Descriptive test names

Coverage Goals:

- Happy path (success cases)
- Failure cases (reverts)
- Edge cases (boundary values)
- Access control verification

Quality Metrics:

- Aim for 80%+ coverage

Common Testing Mistakes

Forgetting setUp()

- Creates non-fresh instances
- Tests depend on each other
- Order-dependent behaviour

Only Testing Happy Path

- Misses failure scenarios
- False sense of security

Poor Test Names

- testFunc() tells you nothing
- testDecrementRevertsWhenZero()
- is much better

Module 5 Recap: Smart Contract Testing

- ✓ Understand why testing is critical
- ✓ Write unit tests in Solidity
- ✓ Use assertions correctly
- ✓ Test failure cases with `vm.expectRevert()`
- ✓ Simulate users with `vm.prank()`
- ✓ Understand fuzz testing basics
- ✓ Professional test structure

Key Takeaway:

"If it's not tested, it's broken"

Next: More advanced testing patterns

Invariant testing, property testing